Docker: From Beginner to Expert — A Comprehensive Tutorial

27 min read · Aug 13, 2024

Suneel Kumar Follow

Listen Share

Docker has revolutionized the way we develop, package, and deploy applications. It provides a standardized way to create lightweight, portable, and self-sufficient containers that can run virtually anywhere. In this comprehensive tutorial, we'll take you from Docker novice to expert, covering everything from basic concepts to advanced techniques.

Photo by Rubaitul Azad on Unsplash

What is Docker?

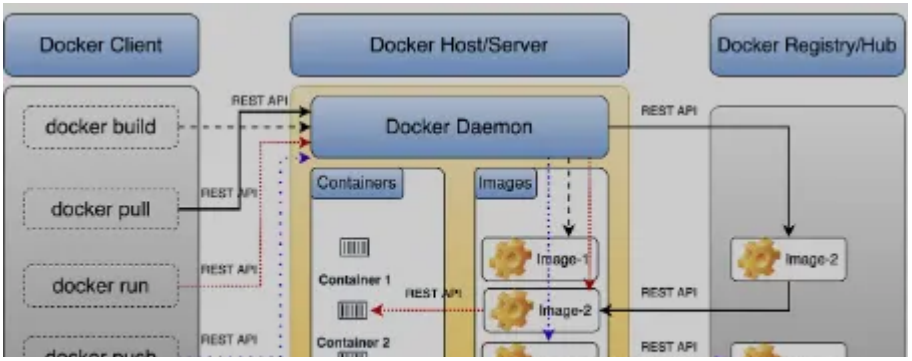
Docker is an open-source platform that automates the deployment, scaling, and management of applications using containerization. Containers are lightweight, standalone executable packages that include everything needed to run a piece of software, including the code, runtime, system tools, libraries, and settings.

Why use Docker?

1. **Consistency**: Docker ensures that your application runs the same way in every environment, from development to production.
2. **Isolation**: Containers are isolated from each other and the host system, improving security and reducing conflicts.
3. **Portability**: Docker containers can run on any system that supports Docker, regardless of the underlying infrastructure.
4. **Efficiency**: Containers share the host OS kernel, making them more lightweight than traditional virtual machines.
5. **Scalability**: Docker makes it easy to scale applications up or down quickly.s

Docker architecture

Docker uses a client-server architecture. The Docker client communicates with the Docker daemon, which does the heavy lifting of building, running, and distributing Docker containers. The client and daemon can run on the same system, or you can connect a Docker client to a remote Docker daemon.



Open in app ↗
Sign up
Sign in
Medium
Search

Key components of Docker architecture:

1. **Docker daemon**: The background service running on the host that manages building, running, and distributing Docker containers.
2. **Docker client**: The command-line tool that allows users to interact with the Docker daemon.
3. **Docker registries**: Repositories that store Docker images. Docker Hub is a public registry that anyone can use.
4. **Docker objects**: Images, containers, networks, volumes, plugins, and other objects.

Now that we have a basic understanding of Docker, let's dive into installation and usage.

2. Installing Docker

Before we can start using Docker, we need to install it on our system. The installation process varies depending on your operating system.

Installing Docker on Ubuntu

1. Update your package index:

```
sudo apt-get update
```

2. Install packages to allow apt to use a repository over HTTPS:

```
sudo apt-get install \
    apt-transport-https \
    ca-certificates \
    curl \
    gnupg-agent \
    software-properties-common
```

3. Add Docker's official GPG key:

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -
```

4. Set up the stable repository:

```
sudo add-apt-repository \
   "deb [arch=amd64] https://download.docker.com/linux/ubuntu \
   $(lsb_release -cs) \
   stable"
```

5. Update the package index again and install Docker:

```
sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io
```

6. Verify that Docker is installed correctly:

```
sudo docker run hello-world
```

Installing Docker on macOS

1. Download the Docker Desktop for Mac installer from the official Docker website.
2. Double-click the downloaded `.dmg` file and drag the Docker icon to your Applications folder.
3. Open Docker from your Applications folder.
4. You may need to provide your system password to complete the installation.

Installing Docker on Windows

1. Download the Docker Desktop for Windows installer from the official Docker website.
2. Double-click the installer to run it.
3. Follow the installation wizard prompts.
4. Once installed, Docker Desktop will start automatically.

Note: For Windows 10 Home, you'll need to install Docker Toolbox instead, which uses VirtualBox to run Docker.

After installation, open a terminal or command prompt and run `docker version` to verify that Docker is installed correctly.

3. Docker Basics

Now that we have Docker installed, let's start with some basic commands and concepts.

Docker CLI

The Docker Command Line Interface (CLI) is the primary way to interact with Docker. Here are some essential commands:

1. Check Docker version:

```
docker version
```

2. Display system-wide information:

```
docker info
```

3. List available Docker commands:

```
docker
```

Docker images

Docker images are read-only templates used to create containers. They contain the application code, libraries, dependencies, tools, and other files needed for an application to run.

1. List locally available images:

```
docker images
```

2. Pull an image from Docker Hub:

```
docker pull ubuntu:latest
```

This command pulls the latest Ubuntu image from Docker Hub.

Running your first container

Let's run our first Docker container using the Ubuntu image we just pulled:

```
docker run -it ubuntu:latest /bin/bash
```

This command does the following:

- `docker run`: Creates and starts a new container
- `-it`: Provides an interactive terminal
- `ubuntu:latest`: Specifies the image to use
- `/bin/bash`: Command to run inside the container

You should now be inside the Ubuntu container. Try running some commands:

```
ls
cat /etc/os-release
```

To exit the container, type `exit` or press `Ctrl+D`.

Container lifecycle

Understanding the container lifecycle is crucial. Here are the main states and commands:

1. Create a container (without starting it):

```
docker create --name mycontainer ubuntu:latest
```

2. Start a container:

```
docker start mycontainer
```

3. Stop a running container:

```
docker stop mycontainer
```

4. docker restart mycontainer

```
docker restart mycontainer
```

5. Pause a running container:

```
docker pause mycontainer
```

6. Unpause a paused container:

```
docker unpause mycontainer
```

7. Remove a container:

```
docker rm mycontainer
```

Note: You can't remove a running container unless you use the `-f` (force) option.

Listing and inspecting containers

1. List running containers:

```
docker ps
```

2. List all containers (including stopped ones):

```
docker ps -a
```

3. Inspect a container:

```
docker inspect mycontainer
```

This command provides detailed information about the container in JSON format.

4. View container logs:

```
docker logs mycontainer
```

Add the `-f` flag to follow the logs in real-time.

Running containers in detached mode

So far, we've run containers in interactive mode. For long-running services, you'll often want to run containers in detached mode:

```
docker run -d --name mywebserver nginx
```

This starts an Nginx web server container in the background. You can still interact with it using other Docker commands.

Now that we've covered the basics, let's dive deeper into working with Docker images.

4. Working with Docker Images

Docker images are the building blocks of containers. Understanding how to work with images is crucial for mastering Docker.

Anatomy of a Docker image

A Docker image is made up of multiple layers. Each layer represents an instruction in the image's Dockerfile. Layers are stacked on top of each other, and each one is the difference between the current state and the previous state.

Finding and pulling images

1. Search for images on Docker Hub:

```
docker search nginx
```

2. Pull an image:

```
docker pull nginx:latest
```

You can specify a particular version by changing the tag (e.g., `nginx:1.19`).

Creating images

There are two main ways to create Docker images:

1. Committing changes made in a container
2. Building from a Dockerfile (preferred method)

Let's look at both methods:

Committing changes

1. Run a container and make some changes:

```
docker run -it ubuntu:latest /bin/bash
apt-get update
apt-get install -y nginx
exit
```

2. Commit the changes to a new image:

```
docker commit <container_id> my-nginx-image:v1
```

Replace `<container_id>` with the ID of the container you just exited.

Building from a Dockerfile

Create a file named `Dockerfile` with the following content:

```
FROM ubuntu:latest
RUN apt-get update && apt-get install -y nginx
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Build the image:

```
docker build -t my-nginx-image:v2 .
```

The `-t` flag tags the image with a name and optional version.

Managing images

1. List images:

```
docker images
```

2. Remove an image:

```
docker rmi my-nginx-image:v1
```

3. Tag an image:

```
docker tag my-nginx-image:v2 my-dockerhub-username/my-nginx-image:v2
```

4. Push an image to Docker Hub:

```
docker push my-dockerhub-username/my-nginx-image:v2
```

Note: You need to be logged in to Docker Hub (`docker login`) before pushing.

Image layers and caching

Docker uses a layer cache to speed up builds. Each instruction in a Dockerfile creates a new layer. If you haven't changed the instruction and its previous layers, Docker will reuse the cached layer.

To see the layers of an image:

```
docker history my-nginx-image:v2
```

Understanding layers helps in creating efficient Dockerfiles, which we'll cover in more detail later.

5. Creating and Managing Docker Containers

Now that we're comfortable with Docker images, let's dive deeper into working with containers.

Running containers

We've already seen the basic `docker run` command, but there are many useful options:

1. Run a container and automatically remove it when it exits:

```
docker run --rm ubuntu:latest echo "Hello, World!"
```

2. Run a container with a custom name:

```
docker run --name my-custom-container ubuntu:latest
```

3. Run a container and publish a port:

```
docker run -p 8080:80 nginx
```

This maps port 8080 on the host to port 80 in the container.

1. Run a container with environment variables:

```
docker run -e MY_VAR=my_value ubuntu:latest env
```

5. Run a container with a limited amount of memory:

```
docker run --memory=512m ubuntu:latest
```

Executing commands in running containers

You can execute commands in a running container using `docker exec`:

```
docker exec -it my-custom-container /bin/bash
```

This opens an interactive shell in the running container.

Copying files between host and container

To copy files between your host system and a container:

1. From host to container:

```
docker cp ./myfile.txt my-custom-container:/path/in/container/
```

2. From container to host:

```
docker cp my-custom-container:/path/in/container/myfile.txt ./
```

Monitoring containers

1. View container resource usage:

```
docker stats
```

2. View processes running in a container:

```
docker top my-custom-container
```

Container resource constraints

Docker allows you to set resource constraints on containers:

1. Limit CPU:

```
docker run --cpus=0.5 ubuntu:latest
```

This limits the container to use at most 50% of a CPU.

2. Set CPU shares (relative weight):

```
docker run --cpu-shares=512 ubuntu:latest
```

The default value is 1024, so this container would receive half the CPU time of a default container.

1. Limit memory and enable swapping:

```
docker run --memory=1g --memory-swap=2g ubuntu:latest
```

This limits the container to 1GB of memory and 2GB of swap.

Updating containers

You can update the configuration of a running container:

1. Update container resources:

```
docker update --cpus=1 --memory=2g my-custom-container
```

2. Rename a container:

```
docker rename my-custom-container my-new-container-name
```

Container restart policies

Docker provides restart policies to automatically restart containers under certain conditions:

```
docker run --restart=always nginx
```

Restart policies:

- `no`: Never restart (default)
- `on-failure[:max-retries]`: Restart only if the container exits with a non-zero status code
- `always`: Always restart the container regardless of the exit status

- `unless-stopped`: Always restart the container unless it was explicitly stopped

Attaching to and detaching from containers

1. Attach to a running container:

```
docker attach my-custom-container
```

2. Detach from a container without stopping it: Press `Ctrl-p` followed by `Ctrl-q`.

Now that we've covered the essentials of working with containers, let's move on to Docker networking.

6. Docker Networking

Docker networking allows containers to communicate with each other and with the outside world. Understanding Docker networking is crucial for building multi-container applications.

Network drivers

Docker supports several network drivers out of the box:

1. **bridge**: The default network driver. It's used for standalone containers that need to communicate.
2. **host**: Removes network isolation between the container and the Docker host.
3. **overlay**: Used for connecting multiple Docker daemons together (used in Swarm mode).
4. **macvlan**: Allows you to assign a MAC address to a container, making it appear as a physical device on your network.
5. **none**: Disables all networking for a container.

Listing and inspecting networks

1. List networks:

```
docker network ls
```

2.Inspect a network:

```
docker network inspect bridge
```

Creating custom networks

Create a custom bridge network:

```
docker network create --driver bridge my-custom-network
```

Connecting containers to networks

1. Connect a running container to a network:

```
docker network connect my-custom-network my-container
```

2. Disconnect a container from a network:

```
docker network disconnect my-custom-network my-container
```

3. Run a new container and connect it to a network:

```
docker run --network my-custom-network nginx
```

Container DNS

Containers on the same user-defined network can resolve each other by name. Let's demonstrate this:

1. Create a network:

```
docker network create my-app-network
```

2. Run two containers on this network:

```
docker run -d --name web --network my-app-network nginx
docker run -d --name db --network my-app-network postgres
```

3. Now, the `web` container can communicate with the `db` container using the hostname `db`.

Port mapping

To make a container's port accessible from the host:

```
docker run -p 8080:80 nginx
```

This maps port 8080 on the host to port 80 in the container.

You can also specify the IP address to bind to:

```
docker run -p 127.0.0.1:8080:80 nginx
```

Network troubleshooting

1. Check container network settings:

```
docker inspect --format '{{json .NetworkSettings.Networks}}' my-container
```

2. Use the `docker exec` command to run network diagnostic tools inside a container:

```
docker exec -it my-container ping google.com
docker exec -it my-container netstat -tuln
```

3. Use the `docker logs` command to check for network-related issues:

```
docker logs my-container
```

4. If you suspect DNS issues, check the container's DNS configuration:

```
docker exec -it my-container cat /etc/resolv.conf
```

Advanced networking topics

1. **Exposing containers**: Use the `--expose` flag to expose ports without publishing them to the host:

```
docker run --expose 80 nginx
```

2. **Link containers** (Note: This is a legacy feature; use user-defined networks instead):

```
docker run --name db postgres
docker run --link db:database nginx
```

3. **MacVLAN networks**: Create a network that allows containers to appear as physical devices on your network:

```
docker network create -d macvlan \
  --subnet=192.168.1.0/24 \
  --gateway=192.168.1.1 \
  -o parent=eth0 my-macvlan-net
```

Understanding Docker networking is crucial for building complex, multi-container applications. Now, let's move on to another important topic: Docker volumes and data management.

7. Docker Volumes and Data Management

Docker containers are ephemeral by nature, meaning any data created inside a container is lost when the container is removed. Docker provides volumes and bind mounts to persist data and share it between containers.

Types of data persistence in Docker

1. **Volumes:** Managed by Docker and stored in a part of the host filesystem.
2. **Bind mounts:** File or directory on the host machine mounted into a container.
3. **tmpfs mounts:** Stored in the host system's memory only.

Working with Docker volumes

1. Create a volume:

```
docker volume create my-vol
```

2. List volumes:

```
docker volume ls
```

3. Inspect a volume:

```
docker volume inspect my-vol
```

4. Remove a volume:

```
docker volume rm my-vol
```

5. Run a container with a volume:

```
docker run -v my-vol:/app/data nginx
```

Bind mounts

Bind mounts allow you to mount a file or directory from the host into a container:

```
docker run -v /host/path:/container/path nginx
```

Use the `--mount` flag for more verbose and explicit volume mounting:

```
docker run --mount type=bind,source=/host/path,target=/container/path nginx
```

Read-only mounts

You can mount volumes or bind mounts as read-only:

```
docker run -v /host/path:/container/path:ro nginx
```

Sharing data between containers

1. Using a named volume:

```
docker run -v shared-data:/app/data --name container1 nginx
docker run -v shared-data:/app/data --name container2 nginx
```

2. Using the `--volumes-from` flag:

```
docker run --name container1 -v /app/data nginx
docker run --volumes-from container1 --name container2 nginx
```

Data backup and restore

1. Backup a volume:

```
docker run --rm -v my-vol:/source -v $(pwd):/backup ubuntu tar cvf /backup/back
```

2. Restore a volume:

```
docker run --rm -v my-vol:/target -v $(pwd):/backup ubuntu tar xvf /backup/back
```

Volume drivers

Docker supports volume drivers to enable volume management on different storage systems:

```
docker volume create --driver local \
    --opt type=nfs \
    --opt o=addr=192.168.1.1,rw \
    --opt device=:/path/to/dir \
    my-nfs-volume
```

This creates an NFS volume using the local driver with specific options.

Understanding Docker volumes and data management is crucial for maintaining data persistence in containerized applications. Next, we'll dive into writing Dockerfiles to create custom images.

8. Writing Dockerfiles

A Dockerfile is a text document containing instructions to build a Docker image automatically. It's a powerful tool for creating reproducible and version-controlled images.

Dockerfile basics

Here's a simple Dockerfile:

```
FROM ubuntu:20.04
RUN apt-get update && apt-get install -y nginx
COPY ./my-nginx.conf /etc/nginx/nginx.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Let's break down the most common Dockerfile instructions:

1. **FROM**: Specifies the base image to use.
2. **RUN**: Executes commands in a new layer on top of the current image.
3. **COPY**: Copies files from the host into the image.
4. **ADD**: Similar to COPY, but can also handle remote URLs and automatically extract compressed files.
5. **EXPOSE**: Informs Docker that the container listens on specified network ports at runtime.
6. **CMD**: Provides defaults for an executing container.
7. **ENTRYPOINT**: Configures a container to run as an executable.

Best practices for writing Dockerfiles

1. **Use official base images** when possible.

2. **Minimize the number of layers** by combining commands:

```
RUN apt-get update && \
    apt-get install -y \
    package1 \
    package2 \
    package3 && \
    rm -rf /var/lib/apt/lists/*
```

3. **Use .dockerignore** to exclude unnecessary files from the build context.

4. **Use multi-stage builds** for smaller final images:

```
FROM golang:1.16 AS builder
WORKDIR /app
COPY . .
RUN go build -o myapp

FROM alpine:3.14
COPY --from=builder /app/myapp /usr/local/bin/myapp
CMD ["myapp"]
```

5. **Set the WORKDIR** to organize your commands:

```
WORKDIR /app
COPY . .
```

6.**Use environment variables** for configuration:

```
ENV APP_HOME /app
WORKDIR $APP_HOME
```

7. **Use LABEL** for metadata:

```
LABEL maintainer="your-email@example.com" \
      version="1.0" \
      description="This is my custom image"
```

Building images from Dockerfiles

To build an image from a Dockerfile:

```
docker build -t my-image:v1 .
```

The `.` at the end specifies the build context (current directory in this case).

Dockerfile instructions in depth

1. **HEALTHCHECK**: Tells Docker how to test if a container is still working:

```
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
  CMD curl -f http://localhost/ || exit 1
```

2. **ARG**: Defines variables that users can pass at build-time:

```
ARG VERSION=latest
FROM ubuntu:${VERSION}
```

3. **VOLUME**: Creates a mount point and marks it as holding externally mounted volumes:

```
VOLUME /app/data
```

4. **USER**: Sets the user name or UID to use when running the image:

```
RUN groupadd -r mygroup && useradd -r -g mygroup myuser
USER myuser
```

Multi-stage builds

Multi-stage builds allow you to use multiple FROM statements in your Dockerfile. Each FROM instruction can use a different base, and each begins a new stage of the build. You can selectively copy artifacts from one stage to another, leaving behind everything you don't want in the final image.

Example of a multi-stage build for a Go application:

```
# Build stage
FROM golang:1.16 AS builder
WORKDIR /app
COPY . .
RUN go mod download
RUN CGO_ENABLED=0 GOOS=linux go build -a -installsuffix cgo -o main .

# Final stage
FROM alpine:3.14
RUN apk --no-cache add ca-certificates
WORKDIR /root/
COPY --from=builder /app/main .
CMD ["./main"]
```

This results in a much smaller final image, as it doesn't include the Go compiler and build tools.

Get Suneel Kumar's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Remember me for faster sign in

Now that we've covered Dockerfiles in depth, let's move on to Docker Compose, which allows you to define and run multi-container Docker applications.

9. Docker Compose

Docker Compose is a tool for defining and running multi-container Docker applications. It uses YAML files to configure application services and performs the creation and start-up of all the containers with a single command.

Docker Compose basics

A simple `docker-compose.yml` file:

```yaml
version: '3'
services:
  web:
    image: nginx:alpine
    ports:
      - "8000:80"
  db:
    image: postgres:12
    environment:
      POSTGRES_PASSWORD: example
```

Key concepts

1. **Services**: Containers that make up your application.
2. **Networks**: How services communicate with each other.
3. **Volumes**: Where services store and share data.

Common Docker Compose commands

1. Start services:

```
docker-compose up
```

2. Start services in detached mode:

```
docker-compose up -d
```

3. Stop services:

```
docker-compose down
```

4. View running services:

```
docker-compose ps
```

5. View logs:

```
docker-compose logs
```

6. Run a command in a service:

```
docker-compose exec web bash
```

Docker Compose file structure

A more complex `docker-compose.yml` example:

```yaml
version: '3.8'

services:
  web:
    build: ./web
    ports:
      - "5000:5000"
```

```
    volumes:
      - ./web:/code
    environment:
      FLASK_ENV: development
    depends_on:
      - db

  db:
    image: postgres:12
    volumes:
      - postgres_data:/var/lib/postgresql/data
    environment:
      POSTGRES_PASSWORD: example

volumes:
  postgres_data:

networks:
  default:
    name: my_app_net
```

Advanced Docker Compose features

1. Scaling services:

```
docker-compose up -d --scale web=3
```

2. Extending services:

```
version: '3.8'

services:
  web:
    extends:
      file: common-services.yml
      service: webapp
    environment:
      - DEBUG=1

  db:
    extends:
      file: common-services.yml
      service: database
```

3. **Using profiles:**

```yaml
version: '3.9'

services:
  web:
    image: nginx:alpine
    profiles: ["frontend", "dev"]

  db:
    image: postgres:12
    profiles: ["backend", "dev"]
```

Start only specific profiles:

```
docker-compose --profile dev up
```

4. **Health checks:**

```yaml
version: '3.8'

services:
  web:
    image: nginx:alpine
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost"]
      interval: 1m30s
      timeout: 10s
      retries: 3
      start_period: 40s
```

Docker Compose is an essential tool for managing multi-container applications. It simplifies the process of defining, running, and scaling services. In the next section, we'll explore Docker Swarm, which allows you to create and manage a cluster of Docker nodes.

10. Docker Swarm

Docker Swarm is Docker's native clustering and orchestration solution. It turns a group of Docker hosts into a single, virtual Docker host, allowing you to scale your application across multiple machines.

Key concepts

1. **Node**: A machine in the Swarm (can be a manager or a worker).
2. **Service**: The definition of tasks to execute on nodes.
3. **Task**: A Docker container and the commands to run inside it.

Setting up a Swarm

1. Initialize a Swarm:

```
docker swarm init --advertise-addr <MANAGER-IP>
```

2. Join a worker node to the Swarm:

```
docker swarm join --token <TOKEN> <MANAGER-IP>:2377
```

3. List nodes in the Swarm:

```
docker node ls
```

Managing services in Swarm

1. Create a service:

```
docker service create --name my_web nginx
```

2. List services:

```
docker service ls
```

3. Scale a service:

```
docker service scale my_web=5
```

4. Update a service:

```
docker service update --image nginx:1.19 my_web
```

5.Remove a service:

```
docker service rm my_web
```

Deploying stacks with Docker Compose

You can use Docker Compose files to deploy entire application stacks to a Swarm:

```
docker stack deploy -c docker-compose.yml my_stack
```

Swarm networking

Swarm mode routing mesh allows each node in the Swarm to accept connections on published ports for any service running in the Swarm, even if there's no task running on that node.

Swarm secrets

Docker Swarm provides a secure way to manage sensitive data:

1. Create a secret:

```
echo "my_secret_data" | docker secret create my_secret -
```

2. Use the secret in a service:

```
docker service create --name nginx --secret my_secret nginx
```

Rolling updates and rollbacks

Swarm supports rolling updates and rollbacks:

```
docker service update --update-parallelism 2 --update-delay 10s my_web
```

To rollback:

```
docker service update --rollback my_web
```

Docker Swarm provides a simple yet powerful way to orchestrate containers across multiple hosts. However, for more complex orchestration needs, you might want to consider Kubernetes, which offers more advanced features and has become the industry standard for container orchestration.

Now that we've covered Docker Swarm, let's move on to Docker security best practices.

11. Docker Security Best Practices

Security is a critical aspect of working with Docker. Here are some best practices to ensure your Docker environment and containers are secure:

1. Keep Docker up to date

Regularly update Docker to the latest version to benefit from security patches and new features.

2. Use official images

Use official images from Docker Hub or trusted sources. These images are typically well-maintained and regularly updated with security patches.

3. Scan images for vulnerabilities

Use tools like Docker Scan, Clair, or Trivy to scan your images for known vulnerabilities:

```
docker scan my-image:latest
```

4. Limit container capabilities

Remove unnecessary capabilities from your containers:

```
FROM ubuntu:20.04
RUN apt-get update && apt-get install -y nginx
RUN setcap cap_net_bind_service=+ep /usr/sbin/nginx
USER www-data
```

5. Use non-root users

Run containers as non-root users whenever possible:

```
FROM ubuntu:20.04
RUN groupadd -r myuser && useradd -r -g myuser myuser
USER myuser
```

6. Use read-only file systems

Mount containers' file systems as read-only:

```
docker run --read-only ubuntu:20.04
```

7. Limit resources

Set resource limits for your containers:

```
docker run --cpus=0.5 --memory=512m ubuntu:20.04
```

8. Use secrets management

Use Docker secrets or a third-party secrets management tool to handle sensitive data:

```
docker secret create my_secret my_secret.txt
docker service create --name nginx --secret my_secret nginx
```

9. Implement network segmentation

Use Docker networks to isolate containers and implement proper network segmentation:

```
docker network create - driver overlay - attachable my_secure_network
docker service create - name nginx - network my_secure_network nginx
```

10. Use security options

Utilize Docker's security options to enhance container isolation:

```
docker run --security-opt=no-new-privileges ubuntu:20.04
```

11. Implement logging and monitoring

Set up proper logging and monitoring for your Docker environment:

```
docker run --log-driver=syslog ubuntu:20.04
```

12. Use Docker Content Trust (DCT)

Enable Docker Content Trust to ensure image integrity:

```
export DOCKER_CONTENT_TRUST=1
docker pull nginx:latest
```

13. Regularly audit your Docker environment

Perform regular security audits of your Docker hosts, images, and containers. Tools like Docker Bench for Security can help:

```
docker run -it --net host --pid host --userns host --cap-add audit_control \
    -e DOCKER_CONTENT_TRUST=$DOCKER_CONTENT_TRUST \
    -v /var/lib:/var/lib \
    -v /var/run/docker.sock:/var/run/docker.sock \
    -v /usr/lib/systemd:/usr/lib/systemd \
    -v /etc:/etc --label docker_bench_security \
    docker/docker-bench-security
```

14. Use multi-stage builds

Multi-stage builds can help reduce the attack surface of your final image by including only necessary components:

```
FROM golang:1.16 AS builder
WORKDIR /app
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -a -installsuffix cgo -o main .

FROM alpine:3.14
RUN apk --no-cache add ca-certificates
COPY --from=builder /app/main .
USER nonroot
CMD ["./main"]
```

15. Implement proper access controls

Use Docker's built-in access control features or third-party solutions to manage user access to Docker resources.

By following these security best practices, you can significantly improve the security posture of your Docker environment. Remember that security is an ongoing process, and it's important to stay informed about the latest security threats and best practices.

Now, let's move on to some advanced Docker techniques that can help you optimize your Docker workflow and improve your containerized applications.

12. Advanced Docker Techniques

As you become more proficient with Docker, you'll want to explore some advanced techniques to optimize your workflow and improve your containerized applications.

1. Custom base images

Create your own base images to standardize your environment:

```
FROM scratch
ADD alpine-minirootfs-3.14.0-x86_64.tar.gz /
CMD ["/bin/sh"]
```

2. Dockerfile heredocs

Use heredocs in Dockerfiles for better readability when dealing with multi-line commands:

```
FROM ubuntu:20.04
RUN <<EOF
apt-get update
apt-get install -y nginx
rm -rf /var/lib/apt/lists/*
EOF
```

3. BuildKit features

Enable BuildKit to use advanced features like multi-platform builds and improved caching:

```
export DOCKER_BUILDKIT=1
```

Then use new Dockerfile syntax:

```
# syntax=docker/dockerfile:1.2
FROM ubuntu:20.04
RUN --mount=type=cache,target=/var/cache/apt \
    apt-get update && apt-get install -y nginx
```

4. Multi-arch builds

Create images that can run on multiple architectures:

```
docker buildx create --use
docker buildx build --platform linux/amd64,linux/arm64 -t myimage:latest .
```

5. Distroless images

Use distroless images for minimal attack surface:

```
FROM golang:1.16 AS builder
WORKDIR /app
COPY . .
RUN CGO_ENABLED=0 go build -o main .

FROM gcr.io/distroless/static
COPY --from=builder /app/main /
CMD ["/main"]
```

6. Docker image squashing

Squash image layers to reduce size and improve pull times:

```
docker build --squash -t myimage:squashed .
```

7. Docker image slimming

Use tools like DockerSlim to automatically optimize your images:

```
docker-slim build myimage:latest
```

8. Custom networks with specific drivers

Create networks with specific drivers for advanced networking scenarios:

```
docker network create --driver macvlan \
  --subnet=192.168.1.0/24 \
  --gateway=192.168.1.1 \
  -o parent=eth0 my_macvlan_net
```

9. Docker plugins

Extend Docker's functionality with plugins:

```
docker plugin install vieux/sshfs
docker volume create -d vieux/sshfs \
  -o sshcmd=user@host:/remote \
  -o password=password \
  sshvolume
```

10. Custom Docker commands

Create custom Docker commands using the plugin system:

```
mkdir -p ~/.docker/cli-plugins
cp ./my-custom-command ~/.docker/cli-plugins/docker-my-custom-command
chmod +x ~/.docker/cli-plugins/docker-my-custom-command
```

11. Docker context

Use Docker contexts to manage multiple Docker environments:

```
docker context create my-remote-docker --docker "host=ssh://user@remote-host"
docker context use my-remote-docker
docker ps  # This will now run on the remote host
```

12. Docker Compose extends

Use the `extends` keyword in Docker Compose to share common configurations:

```yaml
version: '3.8'

services:
  web:
    extends:
      file: common-services.yml
      service: webapp
    environment:
      - DEBUG=1

  db:
    extends:
      file: common-services.yml
      service: database
```

13. Docker Compose anchors and aliases

Use YAML anchors and aliases to reduce repetition in your Docker Compose files:

```yaml
version: '3.8'

x-common-config: &common-config
  restart: always
  environment:
    - COMMON_VAR=value

services:
  web:
    <<: *common-config
    image: nginx:latest

  app:
```

```
    <<: *common-config
    image: my-app:latest
```

These advanced techniques can help you optimize your Docker workflow, improve security, and create more efficient containerized applications. As you continue to work with Docker, you'll likely discover even more advanced techniques and best practices.

Now, let's move on to discussing how to use Docker in production environments.

13. Docker in Production

Using Docker in production requires careful planning and consideration of various factors to ensure reliability, scalability, and security. Here are some key aspects to consider when running Docker in production:

1. Container orchestration

For production environments, you'll typically want to use a container orchestration platform. While Docker Swarm is built into Docker, Kubernetes has become the de facto standard for container orchestration in production environments.

Kubernetes offers:

- Advanced scheduling and scaling
- Self-healing capabilities
- Rolling updates and rollbacks
- Service discovery and load balancing
- Configuration management

2. Monitoring and logging

Implement comprehensive monitoring and logging for your containerized applications:

- Use tools like Prometheus and Grafana for monitoring
- Implement centralized logging with the ELK stack (Elasticsearch, Logstash, Kibana) or similar solutions

- Use Docker's logging drivers to send logs to your centralized logging system:

```
docker run --log-driver=fluentd --log-opt fluentd-address=localhost:24224 my-ap
```

3. CI/CD pipelines

Integrate Docker into your CI/CD pipelines:

- Automatically build and test Docker images on each commit
- Use multi-stage builds to optimize your build process
- Implement vulnerability scanning in your pipeline
- Automate deployments to your orchestration platform

4. High availability

Design your applications and infrastructure for high availability:

- Use multiple replicas for each service
- Implement proper health checks
- Use rolling updates to minimize downtime during deployments
- Utilize load balancers to distribute traffic across containers

5. Persistent storage

Manage persistent storage carefully in production:

- Use volume drivers appropriate for your infrastructure (e.g., AWS EBS, GCE Persistent Disk)
- Implement proper backup and recovery strategies for your volumes
- Consider using stateless containers where possible to simplify scaling and management

6. Networking

Implement a robust networking strategy:

- Use overlay networks for multi-host networking
- Implement network policies to control traffic between services
- Use service discovery mechanisms provided by your orchestration platform

7. Security

Implement multiple layers of security:

- Use a container-specific OS like CoreOS or RancherOS to reduce attack surface
- Implement strong access controls and authentication mechanisms
- Use network segmentation to isolate sensitive services
- Regularly update and patch your containers and host systems
- Implement runtime security monitoring

8. Resource management

Properly manage resources to ensure stability and efficiency:

- Set appropriate resource limits for your containers
- Use resource quotas at the namespace level in Kubernetes
- Implement autoscaling based on resource usage or custom metrics

9. Image management

Establish a robust image management strategy:

- Use a private registry for production images
- Implement proper versioning for your images
- Use image scanning tools to detect vulnerabilities
- Implement policies to prevent the use of outdated or vulnerable images

10. Backup and disaster recovery

Implement comprehensive backup and disaster recovery strategies:

- Regularly backup your data volumes
- Implement cross-region or cross-datacenter replication for critical data

- Document and regularly test your disaster recovery procedures

11. Performance optimization

Optimize your containers and infrastructure for performance:

- Use performance monitoring tools to identify bottlenecks
- Optimize your Dockerfiles for smaller, more efficient images
- Use appropriate storage drivers for your workload
- Consider using caching mechanisms like Redis for frequently accessed data

12. Cost management

Implement strategies to manage and optimize costs:

- Use appropriate instance types for your workload
- Implement autoscaling to match resources with demand
- Use spot instances or preemptible VMs where appropriate
- Regularly review and optimize your resource usage

13. Documentation and runbooks

Maintain comprehensive documentation and runbooks:

- Document your architecture and deployment procedures
- Create runbooks for common operations and troubleshooting scenarios
- Keep your documentation up-to-date as your systems evolve

Running Docker in production requires careful planning and ongoing management. By considering these aspects and implementing best practices, you can create a robust, scalable, and efficient production environment for your containerized applications.

Now, let's move on to troubleshooting and debugging Docker containers, which is an essential skill for managing Docker in both development and production environments.

14. Troubleshooting and Debugging

When working with Docker, you'll inevitably encounter issues that require troubleshooting and debugging. Here are some techniques and tools to help you diagnose and resolve problems with Docker containers:

1. Checking container logs

The first step in troubleshooting is often to check the container logs:

```
docker logs <container_id>
```

For continuous log output:

```
docker logs -f <container_id>
```

2. Inspecting containers

Get detailed information about a container:

```
docker inspect <container_id>
```

You can use Go templates to filter specific information:

```
docker inspect --format '{{.State.Status}}' <container_id>
```

3. Executing commands in running containers

Run commands inside a running container for debugging:

```
docker exec -it <container_id> /bin/bash
```

4. Checking resource usage

View resource usage statistics for running containers:

```
docker stats
```

5. Debugging network issues

Inspect the container's network settings:

```
docker network inspect <network_name>
```

Use network troubleshooting tools inside the container:

```
docker exec -it <container_id> ping google.com
```

6. Analyzing filesystem changes

View changes made to a container's filesystem:

```
docker diff <container_id>
```

7. Debugging build issues

Use the `--progress=plain` flag to see detailed build output:

```
docker build --progress=plain -t myimage .
```

8. Using Docker events

Monitor Docker events in real-time:

```
docker events
```

9. Checking Docker daemon logs

On Linux systems, check the Docker daemon logs:

```
journalctl -u docker.service
```

10. Using debug mode

Run the Docker daemon in debug mode:

```
dockerd -D
```

11. Debugging multi-container applications

For Docker Compose applications, use:

```
docker-compose logs
docker-compose ps
docker-compose top
```

12. Using third-party tools

Several third-party tools can help with debugging:

- Sysdig: For system-level visibility
- cAdvisor: For container resource usage and performance characteristics
- Weave Scope: For visualizing your Docker containers, images, and hosts

13. Debugging kernel issues

For issues that might be related to the kernel, check:

```
dmesg | tail
```

14. Using Docker's experimental features

Enable experimental features for additional debugging capabilities:

```
docker version -f '{{.Server.Experimental}}'
```

15. Debugging Docker Swarm

For Swarm-specific issues:

```
docker node ls
docker service logs <service_name>
docker service ps <service_name>
```

Common issues and solutions

1. **Container won't start**

- Check logs: `docker logs <container_id>`
- Verify the image exists: `docker images`
- Check for port conflicts: `docker port <container_id>`

1. **Container exits immediately**

- Ensure the container has a running process: Check the CMD or ENTRYPOINT in the Dockerfile
- Use `docker run -it` to start the container interactively and investigate

1. **Cannot connect to a container's exposed port**

- Verify the port mapping: `docker port <container_id>`

- Check if the application inside the container is binding to 0.0.0.0 instead of localhost

1. **Out of disk space**

- Remove unused containers, images, and volumes: `docker system prune`
- Increase the size of the Docker host's filesystem

1. **Container is running but not responding**

- Check resource usage: `docker stats <container_id>`
- Inspect logs for any error messages: `docker logs <container_id>`

1. **Network connectivity issues between containers**

- Verify containers are on the same network: `docker network inspect <network_name>`
- Check for any network policies or firewall rules that might be blocking traffic

Remember, effective troubleshooting often involves a combination of these techniques. As you gain experience, you'll develop intuition about which approach is most likely to yield results for different types of issues.

Now, let's explore the broader Docker ecosystem and some useful tools that can enhance your Docker workflow.

15. Docker Ecosystem and Tools

The Docker ecosystem is rich with tools and projects that extend Docker's functionality and integrate with various aspects of software development and deployment. Here's an overview of some key tools and projects in the Docker ecosystem:

1. Container Orchestration

- **Kubernetes**: The de facto standard for container orchestration
- **Docker Swarm**: Docker's native clustering and orchestration solution
- **Nomad**: A flexible workload orchestrator to deploy and manage containers and non-containerized applications

2. Container Registries

- **Docker Hub:** Docker's public container registry
- **Amazon Elastic Container Registry (ECR):** AWS's fully-managed Docker container registry
- **Google Container Registry:** Google Cloud's Docker image registry
- **Azure Container Registry:** Microsoft Azure's container registry service
- **Harbor:** An open-source container registry project

3. Continuous Integration/Continuous Deployment (CI/CD)

- **Jenkins:** An open-source automation server with Docker pipeline support
- **GitLab CI/CD:** GitLab's built-in CI/CD system with excellent Docker support
- **CircleCI:** A CI/CD platform with first-class Docker support
- **Travis CI:** A hosted CI service that supports Docker-based builds
- **GitHub Actions:** GitHub's automation platform with Docker support

4. Monitoring and Logging

- **Prometheus:** An open-source monitoring and alerting toolkit
- **Grafana:** A popular open-source platform for monitoring and observability
- **ELK Stack (Elasticsearch, Logstash, Kibana):** A powerful logging and analytics suite
- **Fluentd:** An open-source data collector for unified logging layer
- **cAdvisor:** Analyzes resource usage and performance characteristics of running containers

5. Security

- **Clair:** Open source project for static analysis of vulnerabilities in Docker containers
- **Anchore Engine:** A service that analyzes Docker images and applies user-defined acceptance policies
- **Trivy:** A simple and comprehensive vulnerability scanner for containers

- **Falco**: Cloud-native runtime security project

6. Development Tools

- **Docker Compose**: Defines and runs multi-container Docker applications
- **Visual Studio Code with Docker extension**: Provides IntelliSense for Dockerfiles and docker-compose.yml files
- **Portainer**: Web-based management UI for Docker environments
- **Dive**: A tool for exploring a Docker image, layer contents, and discovering ways to shrink the size of your Docker image

7. Networking

- **Weave Net**: Creates a virtual network that connects Docker containers across multiple hosts
- **Calico**: An open-source networking and network security solution for containers
- **Cilium**: Provides and secures network connectivity between application containers

8. Storage

- **Rexray**: Provides persistent storage for Docker containers
- **Flocker**: An open-source container data volume manager
- **Longhorn**: Cloud-native distributed block storage system for Kubernetes

9. Service Discovery

- **Consul**: A service mesh solution providing service discovery, configuration, and segmentation functionality
- **etcd**: A distributed, reliable key-value store for the most critical data of a distributed system

10. Container Management Platforms

- **Rancher**: An open-source platform for managing Docker in production
- **OpenShift**: Red Hat's container application platform based on Kubernetes
- **Docker Enterprise**: Docker's commercial platform for container-based application development

11. Serverless Frameworks for Containers

- **OpenFaaS**: Serverless Functions Made Simple for Docker & Kubernetes
- **Knative**: Kubernetes-based platform to deploy and manage modern serverless workloads

12. Local Development Environments

- **Docker Desktop**: An easy-to-install application for your Mac or Windows environment
- **Minikube**: Runs a single-node Kubernetes cluster inside a VM on your laptop

13. Image Building

- **BuildKit**: Docker's next-generation image building backend
- **Kaniko**: Build container images in Kubernetes without depending on a Docker daemon
- **Buildah**: A tool that facilitates building OCI container images

14. Configuration Management

- **confd**: Manage local application configuration files using templates and data from etcd or consul
- **envsubst**: Substitutes environment variables in shell format strings

15. Testing

- **Test Kitchen**: A test harness tool to execute your configured code on one or more platforms in isolation
- **Container Structure Test**: A framework to validate the structure of a container image

These tools and projects represent just a fraction of the rich Docker ecosystem. As you become more proficient with Docker, you'll likely find specific tools that fit well with your workflow and project requirements.

Now, let's wrap up this comprehensive tutorial with some concluding thoughts and suggestions for next steps in your Docker journey.

16. Conclusion and Next Steps

Congratulations! You've completed this comprehensive Docker tutorial, which has taken you from the basics all the way to advanced techniques and an overview of the broader Docker ecosystem. Let's recap what we've covered and suggest some next steps for your Docker journey.

Recap

In this tutorial, we've covered:

1. Introduction to Docker and its core concepts
2. Installing Docker on various platforms
3. Working with Docker images and containers
4. Docker networking and volumes
5. Writing Dockerfiles and building custom images
6. Using Docker Compose for multi-container applications
7. Docker Swarm for container orchestration
8. Security best practices for Docker
9. Advanced Docker techniques
10. Using Docker in production environments
11. Troubleshooting and debugging Docker containers
12. An overview of the Docker ecosystem and related tools

Next Steps

To continue your Docker journey and deepen your expertise, consider the following steps:

1. **Practice, practice, practice**: The best way to become proficient with Docker is to use it regularly. Try to incorporate Docker into your development workflow and experiment with containerizing different types of applications.
2. **Dive deeper into orchestration**: While we covered Docker Swarm, consider learning Kubernetes, which has become the industry standard for container orchestration. The Kubernetes official documentation is a great place to start.

3. **Explore cloud platforms**: Learn how to use Docker with major cloud platforms like AWS, Google Cloud, or Azure. Each platform has its own container services that integrate well with Docker.

4. **Contribute to open-source projects**: Many Docker-related projects are open-source. Contributing to these projects can be a great way to learn more and give back to the community.

5. **Stay updated**: Docker and its ecosystem are evolving rapidly. Follow Docker blogs, attend conferences or webinars, and join Docker community forums to stay up-to-date with the latest developments.

6. **Certification**: Consider pursuing Docker certification, such as the Docker Certified Associate (DCA) exam, to validate your skills and knowledge.

7. **Explore related technologies**: Look into related technologies like serverless computing, service meshes, and GitOps to understand how they complement and extend container-based architectures.

8. **Focus on security**: Deepen your understanding of container security. Explore tools and best practices for securing containerized applications throughout the development lifecycle.

9. **Learn about monitoring and observability**: Dive deeper into monitoring and observability solutions for containerized environments. Understanding how to effectively monitor and troubleshoot containerized applications is crucial for production use.

10. **Experiment with different application architectures**: Try containerizing different types of applications, from monoliths to microservices, to understand the strengths and challenges of each approach.

Remember, Docker is just a tool, albeit a powerful one. The ultimate goal is to use Docker to improve your development workflow, enhance application deployment, and solve real-world problems. As you continue your Docker journey, always keep in mind how Docker can be applied to address the specific challenges you face in your projects and organization.

Docker has transformed the way we develop, ship, and run applications. By mastering Docker, you're equipping yourself with a valuable skill set that's in high

demand in the software industry. Keep learning, stay curious, and happy containerizing!

Docker | Docker Compose | DevOps | Programming | Networking

Follow

Written by Suneel Kumar

6.8K followers · 10 following

>>>please visit https://www.linkedin.com/in/suneel-kumar-nodejs/

Responses (5)

Write a response

What are your thoughts?

See all responses

More from Suneel Kumar

Suneel Kumar

20 Advanced SQL Techniques with Practical Examples

SQL (Structured Query Language) is a powerful language for managing and manipulating relational databases. While most developers are...

Mar 15, 2024 1

Suneel Kumar

API Design: From Basics to Best Practices

Application Programming Interfaces (APIs) have become an integral part of modern software development, enabling different systems and...

Mar 13, 2024 2

Suneel Kumar

Message Queue in Node.js with BullMQ and Redis

A message queue is a mechanism for communicating between different components of an application in an asynchronous way. It allows...

Sep 27, 2023 7

Suneel Kumar

Unlock Advanced AI Models in Opencode: The opencode-antigravity-auth Plugin Revolution

Access Gemini 3 Pro, Claude Opus 4.5, and More with Higher Rate Limits Using Google's Antigravity

Jan 12 1

See all from Suneel Kumar

Recommended from Medium

In Women in Technology by Alina Kovtun

Stop Memorizing Design Patterns: Use This Decision Tree Instead

Choose design patterns based on pain points: apply the right pattern with minimal over-engineering in any OO language.

Jan 29 64

In Level Up Coding by Kusireddy

I Stopped Using ChatGPT for 30 Days. What Happened to My Brain Was Terrifying.

91% of you will abandon 2026 resolutions by January 10th. Here's how to be in the 9% who actually win.

Dec 28, 2025 430

In Generative AI by Adham Khaled

Stanford Just Killed Prompt Engineering With 8 Words (And I Can't Believe It Worked)

ChatGPT keeps giving you the same boring response? This new technique unlocks 2× more creativity from ANY AI model—no training required...

Oct 20, 2025 685

In Write A Catalyst by DevOps voice

🧰 COMPLETE DEVOPS CHEAT SHEET HANDBOOK

A practical, example-driven DevOps command reference covering Linux, Git, Docker, Kubernetes, Terraform, Ansible, CI/CD, Monitoring...

Jan 15 3

Jacob Bennett

The 5 paid subscriptions I actually use in 2026 as a Staff Software Engineer

Tools I use that are (usually) cheaper than Netflix

Jan 19 103

In CodeX by Pawan Natekar

Linux Is Hard—Until You Understand These 7 Things

The beginner mistakes nobody warns you about

Dec 14, 2025 50

See more recommendations